

# テクノロジー系試験範囲

## 1. 基礎理論

### 1-1. 基礎理論

離散数学/応用数学/情報に関する理論/通信に関する理論/継続・制御に関する理論

### 1-2. アルゴリズムとプログラミング

データ構造/アルゴリズム/プログラミング/プログラム言語/その他の言語

## 2. コンピュータシステム

### 2-1. コンピュータ構成要素

プロセッサ/メモリ/バス/入出力デバイス/入出力装置

### 2-2. システム構成要素

システムの構成/システムの評価指標

### 2-3. ソフトウェア

オペレーティングシステム/ミドルウェア/ファイルシステム/開発ツール/オープンソースソフトウェア

### 2-4. ハードウェア

ハードウェア

# テクノロジー系試験範囲

## 3. 技術要素

### 3-1. ヒューマンインターフェース

ヒューマンインターフェース技術/インタフェース設計

### 3-2. マルチメディア

マルチメディア技術/マルチメディア応用

### 3-3. データベース

データベース方式/データベース設計/データ操作/トランザクション処理/データベース応用

### 3-4. ネットワーク

ネットワーク方式/データ通信と制御/通信プロトコル/ネットワーク管理/ネットワーク応用

### 3-5. セキュリティ

情報セキュリティ/情報セキュリティ管理/セキュリティ技術評価/情報セキュリティ対策/情報セキュリティ実装技術

## 4. 開発技術

### 4-1. システム開発技術

システム要件定義/システム方式設計/ソフトウェア要件定義/ソフトウェア方式設計・ソフトウェア詳細設計/ソフトウェア構築/ソフトウェア結合・ソフトウェア適格性確認テスト/ソフトウェア結合・ソフトウェア適格性確認テスト/システム結合・システム適格性確認テスト/導入/受入れ支援/保守・廃棄

### 4-2. ソフトウェア開発管理技術

開発プロセス・手法/知的財産権運用管理/開発環境管理/構成管理・変更管理

# コンピュータシステム

## ソフトウェア オペレーティングシステム、ミドルウェア、 ファイルシステム

- 目標：**
- OS の種類, 特徴, 機能, 構成を理解し, 担当する事項に適用する。
  - ジョブ管理, タスク管理, 記憶管理など OS の代表的な機能について, 管理の仕組みを理解し, 担当する事項に適用する。
  - 代表的なミドルウェアの役割, 機能を修得する
  - ファイルを階層化して管理するディレクトリの種類, 特徴, ファイル管理の仕組みを修得する。
  - ファイルシステムの種類, 特徴を修得する。
  - ファイル編成, アクセス手法, 検索手法, バックアップ方式を修得する

# OSの種類

|                       |                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| Windows               | Microsoft製のWindowsコンピュータで利用する                                                                                  |
| MacOS                 | Unixがベースになっている。Apple製のMacコンピュータで利用する                                                                           |
| Unix                  | 1969年から開発。研究・企業のワークステーションで利用。マルチユーザ、マルチプロセス。ディレクトリやプロセスをファイルと同様に扱える。 <b>デーモン</b> と呼ばれるバックグラウンドプロセスがサービスを提供している |
| Linux(Debian, Redhat) | 1991年から開発が始まったUnix系のOS。オープンソースソフトウェアでサーバOSとして用いられる。Debian系、Redhat系が有名                                          |
| Android, IOS          | 主にスマートフォンの端末で利用されるOS。                                                                                          |

# OSの起動

**BIOS**・・・OSが起動する前に、立ち上がる。ハードウェアの制御を行い、起動するOSを選択して、OSをメモリに読み込む。PCの電源を入れた後の起動順は、BIOS→OS→各ソフトウェア

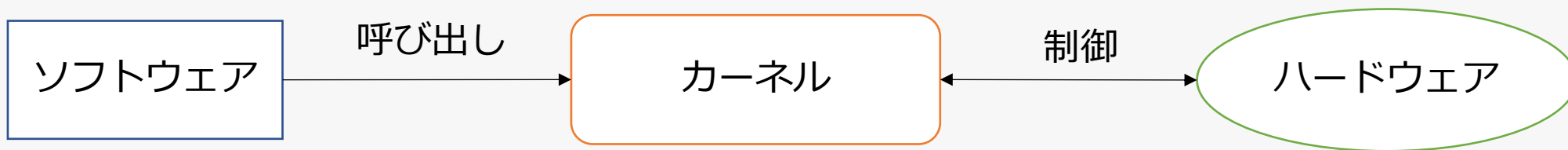
**マルチブート**・・・ハードウェア上に複数のOSをインストールして選択して起動できるようにすること

## OSの役割

|                    |                                                                |
|--------------------|----------------------------------------------------------------|
| <b>ユーザの管理</b>      | ユーザの登録・抹消。ユーザ別のアクセス権の管理。                                       |
| <b>ハードウェア資源の管理</b> | CPU,メモリなどのハードウェア資源を効率的に使えるようにする                                |
| <b>タスクの管理</b>      | コンピュータが実行する処理（タスク）、立ち上げるソフトウェアを制御する                            |
| <b>ファイルの管理</b>     | ファイルの保存、削除など管理する                                               |
| <b>周辺機器の管理</b>     | マウス、プリンタなどの周辺機器を接続して使用できるようにする。周辺機器を利用するには <b>デバイスドライバ</b> が必要 |

# カーネル

osはカーネルとそれ以外の部分に分けることができ、カーネルはその中核。コンピュータのリソースの管理、ハードウェアの制御を行う



|            |                                                   |
|------------|---------------------------------------------------|
| モノリシックカーネル | さまざまな機能をカーネルに持たせる。プロセス管理、メモリ管理、ファイルシステム管理、デバイス管理等 |
| マイクロカーネル   | プロセス管理等最小限の機能をカーネルに持たせる                           |

# メモリ管理



**空きメモリ領域確保のアルゴリズム:** メモリが必要となった場合に、空いているメモリを確保する

**ファーストフィット:** 一番最初に見つかった空き領域を確保する

(100KBメモリが必要で、空き領域が50KB, 150KB, 120KBの順で見つかった場合に150KBを利用する)

**ベストフィット:** 必要な最小の領域を確保する

(100KBメモリが必要で、空き領域が50KB, 150KB, 120KBの順で見つかった場合に120KBを利用する)

**オーバレイ:** プログラムが大きくてメモリに入りきらないときに、プログラムを分割して、その時に必要な分の容量をメモリに読み込んで実行する処理

## 応用情報技術者平成27年春期 午前問18

500kバイトの連続した空き領域に、複数のプログラムモジュールをオーバレイ方式で読み込んで実行する。読み込み順序Aと読み込み順序Bにおいて、最後の120kバイトのモジュールを読み込む際、読み込み可否の組合せとして適切なものはどれか。ここで、数値は各モジュールの大きさをkバイトで表したものであり、モジュールを読み込む領域は、ファーストフィット方式で求めることとする。

[読み込み順序A]

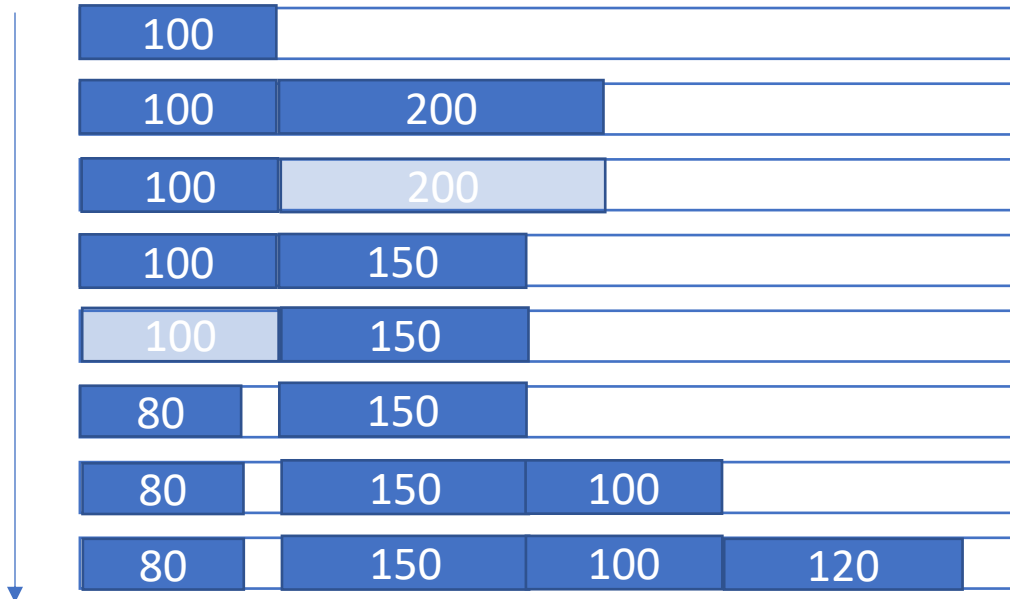
100→200→200解放→150→100解放→80→100→120

[読み込み順序B]

200→100→150→100解放→80→200解放→100→120

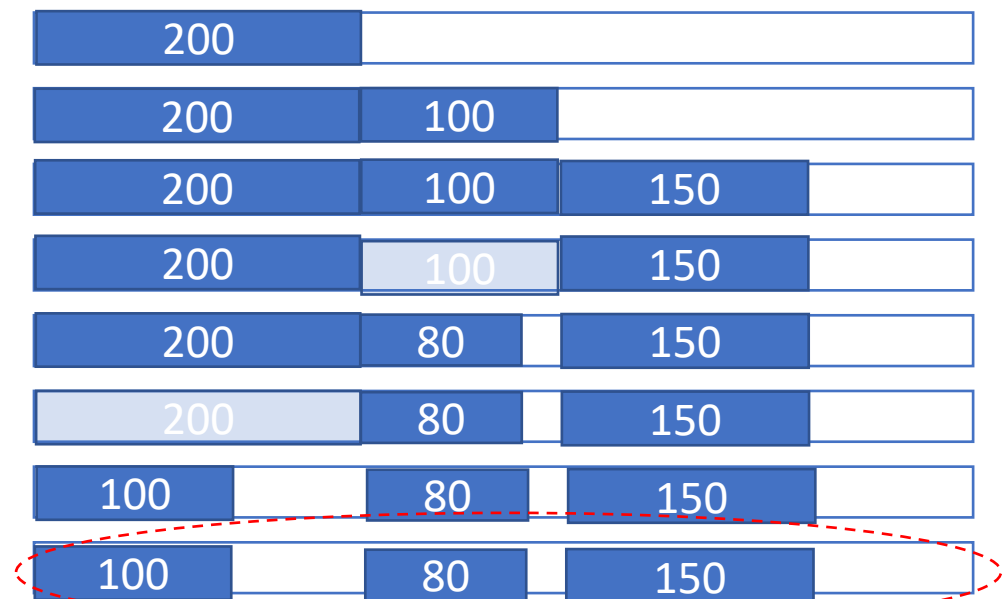
|   | 読み込み順序 A | 読み込み順序 B |
|---|----------|----------|
| ア | 読み込み可能   | 読み込み可能   |
| イ | 読み込み可能   | 読み込み不可能  |
| ウ | 読み込み不可能  | 読み込み可能   |
| エ | 読み込み不可能  | 読み込み不可能  |

読み込み順序A



読み込み順序B

120k読み込む領域がない



# 仮想メモリ

**ページング**: 仮想メモリを使用する際のメモリ管理方式。ページと呼ばれる単位でメモリを管理して、仮想的なアドレスを用いることでハードディスク上にメモリ領域を確保する

**ページフォールト**: 実行時に必要な領域が主記憶上に存在しないこと

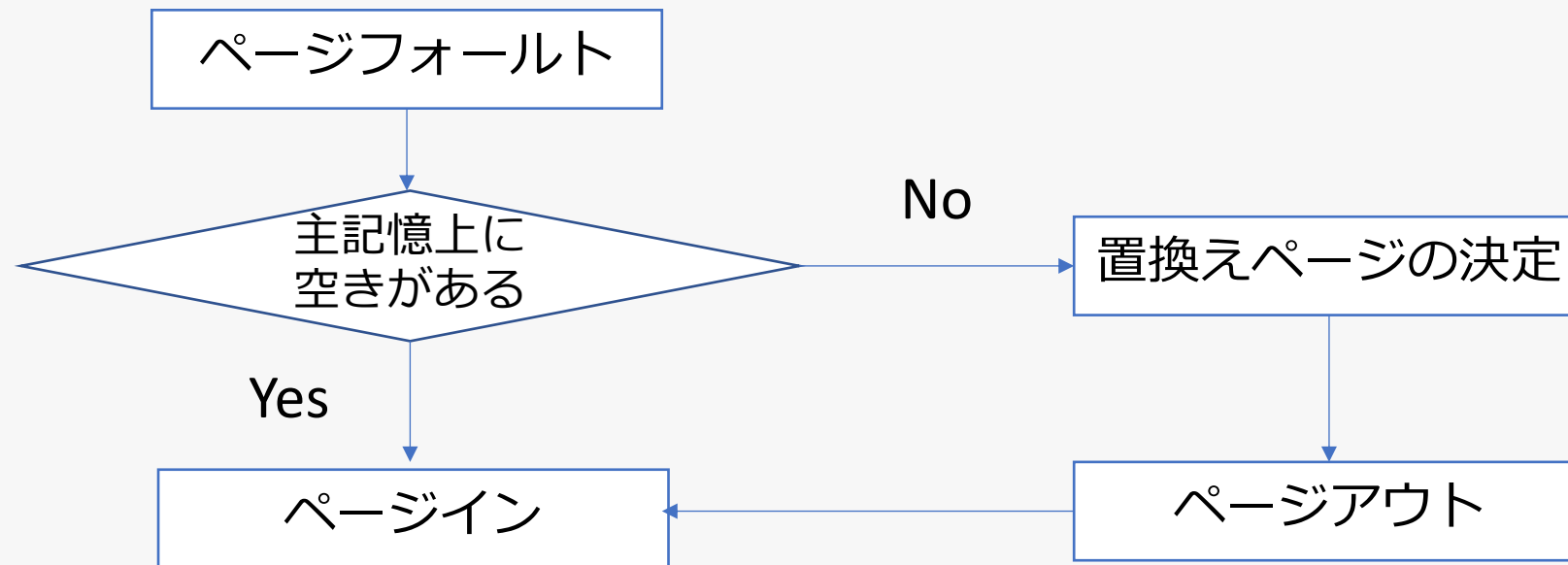
**ページイン**: 仮想メモリからメモリにページをロードすること

**ページアウト**: 主記憶にあるページを仮想メモリに移すこと

**ページリプレースメント**: ページイン、ページアウトのこと

**デマンドページング**: ページフォールトが発生した場合に、仮想メモリからメモリを読み込む（ページイン）

**プリページング**: ページフォールトが発生する前に、あらかじめ必要な容量を予測しておいてページインをする



# ページ置換えアルゴリズム

ページアウトするページを選択するアルゴリズム

|                           |                                |
|---------------------------|--------------------------------|
| FIFO(First-In First-Out)  | 最も長い間主記憶に存在するページ（最も早く読み込んだページ） |
| LIFO(Last-In First-Out)   | 後入れ先出し。最後に読み込んだページ。            |
| LRU(Least Recently Used)  | 最後に参照されてから経過時間が一番長いページ         |
| LFU(Least Frequency Used) | 最も使用頻度の少ないページ                  |

**スラッシング:** プログラムの多重度が増加して、ページの置き換えが多発しシステムの処理効率が低下すること。プロセスの処理にオーバーヘッドが生じるため、CPU使用率は低下する

## 応用情報技術者平成27年秋期 午前問17

デマンドページング方式による仮想記憶の利点はどれか。

ア.実際にアクセスが行われたときにだけ主記憶にロードするので、無駄なページをロードしなくて済む。

→正しい。デマンドページング方式はページフォールト発生時のみロードするので無駄がない

イ.主記憶に対する仮想記憶の容量比を大きくするほど、ページフォールトの発生頻度を低くできる。

→仮想記憶の容量比が増えるほどページフォールトが発生しやすくなります

ウ.プロセスが必要とするページを前もって主記憶にロードするので、補助記憶へのアクセスによる遅れを避けることができる。

→プリページングです

エ.ページフォールトの発生頻度が極端に高くなっても、必要な場合にしかページを読み込まないのでスラッシング状態を回避できる。

→ページフォールトの回数が高くなるとスラッシング状態になりやすくなります。

### 応用情報技術者平成29年春期 午前問16

4ブロック分のキャッシュメモリC0～C3が表に示す状態である。ここで、新たに別のブロックの内容をキャッシュメモリにロードする必要があるとき、C2のブロックを置換の対象とするアルゴリズムはどれか。

ア.FIFO

→FIFOでは、一番最初にロードされたC0です

イ.LFU

→LFUは最も参照回数の少なりC1です

ウ.LIFO

→LIFOでは、最も最後にロードされたC3です

エ.LRU

→LRUでは、最も最近利用していないC2です。正しい

| キャッシュメモリ | ロード時刻（分：秒） | 最終参照時刻（分：秒） | 参照回数 |
|----------|------------|-------------|------|
| C0       | 0:00       | 0:08        | 10   |
| C1       | 0:03       | 0:06        | 1    |
| C2       | 0:04       | 0:05        | 3    |
| C3       | 0:05       | 0:10        | 5    |

### 基本情報技術者平成24年春期 午前問21

仮想記憶システムにおいて主記憶の容量が十分でない場合、プログラムの多重度を増加させるとシステムのオーバヘッドが増加し、アプリケーションのプロセッサ使用率が減少する状態を表すものはどれか。

ア.スラッシング

→正しい。スラッシングの説明です

イ.フラグメンテーション

→フラグメンテーションは、ディスクの領域が断片化すること

ウ.ページング

→ページングは、仮想メモリを使用する際のメモリ管理方式です。

エ.ボトルネック

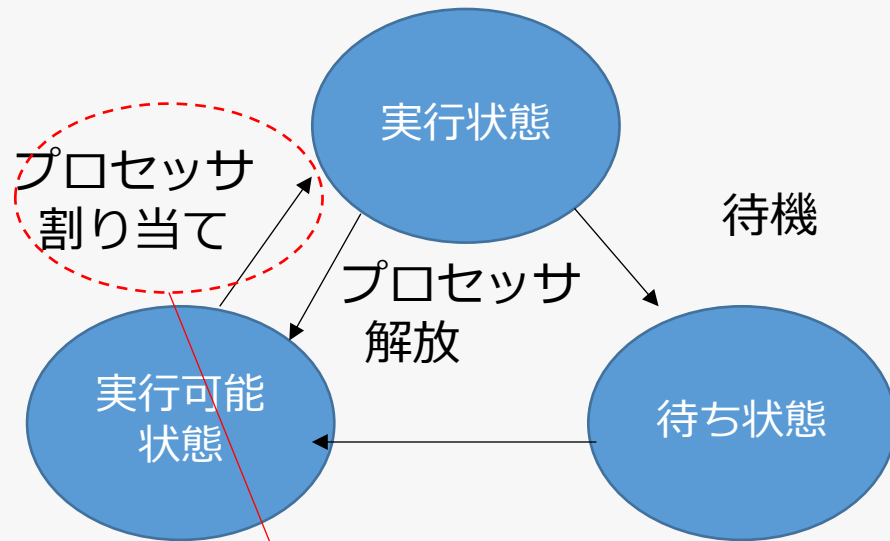
→ボトルネックは、パフォーマンス低下の原因となっている部分です

# プロセスの状態遷移

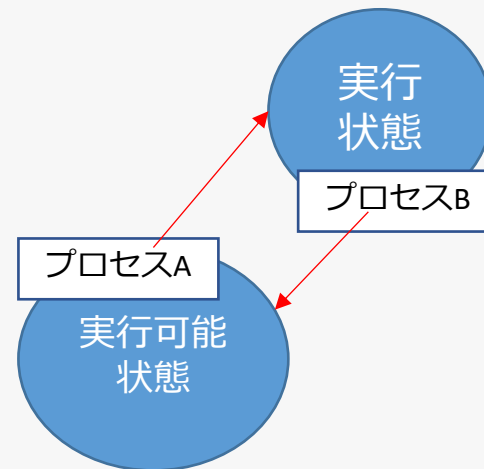
プロセスはプログラムの実行単位。osはプロセスを制御する。その制御を理解するために状態遷移を用います

|        |                                          |
|--------|------------------------------------------|
| 実行状態   | 実行に必要なプロセッサが割り当てられ、CPUが稼働しプロセスが実行されている状態 |
| 実行可能状態 | リソースのプロセッサがあれば、すぐにプロセスを実行できる状態           |
| 待ち状態   | 実行可能状態に移るための事象発生（入出力の完了等）を待っている状態        |

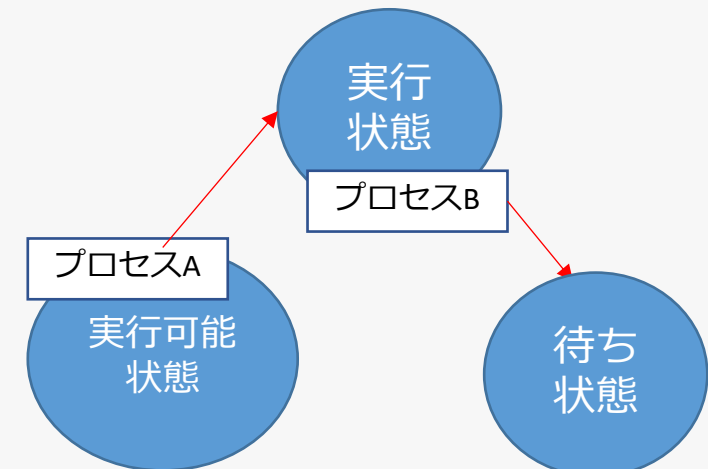
## コンテキスト切替え



**ディスパッチ:** 実行可能状態のプロセスにプロセッサを割り当てて、実行状態に移すこと。これを行う機構を**ディスパッチャ**と言う



**プリエンプティブ:**  
実行中のプロセスからプロセッサの割り当てを強制的に取り上げ実行可能状態のプロセスに渡すこと



**ノンプリエンプティブ:**  
強制的に割り当てを取り上げずに、実行中のプロセスがプロセッサの解放をのを待ち、別のプロセスに割り当てる

# スケジューリング方式

実行可能状態のプロセスの待ち行列をどのように形成するのか、どのようにディスパッチするのか決める方式

|                        |                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------|
| ラウンドロビン方式              | 各タスクに一定時間のCPU使用時間( <b>タイムクォンタム</b> )を渡し、その時間を超えたものは待ち行列の最後列に回して実行可能状態に移る                               |
| 優先度順方式                 | 各タスクにあらかじめ優先度を設定して優先度に合わせて待ち行列を生成する。この際、優先度の低いプロセスも処理されるように一定時間たった場合に優先度を上げる処理が取られる ( <b>動的優先順制御</b> ) |
| 到着順方式                  | 各タスクの到着順 (作成された順) に、待ち行列を作成し先頭からプロセッサを割り当てる。                                                           |
| SJF(Short Job First)方式 | より短い時間で処理が終了する処理から順に待ち行列を生成する。処理時間の長い処理は長時間待つことになる                                                     |
| 多重待ち行列方式               | ラウンドロビン方式と優先度順方式を組み合わせる。優先度順方式の待ち行列とラウンドロビン方式のタスク制御をする                                                 |
| フィードバック待ち行列方式          | ラウンドロビン方式と優先度順方式を組み合わせる。優先度順方式の待ち行列とラウンドロビン方式のタスク制御をする。この際、タイムクォンタムを使い切ったタスクの優先度を下げる                   |

### 基本情報技術者平成31年春期 午前問16

タスクのディスパッチの説明として、適切なものはどれか。

ア.各タスクの実行順序を決定すること

→誤り。これはスケジューリングの説明です

イ.実行可能なタスクに対してプロセッサの使用権を割り当てること

→正しい。ディスパッチャはタスクにプロセッサの使用権を割り当てることです

ウ.タスクの実行に必要な情報であるコンテキストのこと

→誤り。これは、TCB(Task Control Block)と呼ばれています

エ.一つのプロセッサで複数のタスクを同時に実行しているかのように見せかけた状態のこと

→誤り。マルチタスクの説明です

### 基本情報技術者平成24年秋期 午前問17

タスクスケジューリング方式の説明のうち、特定のタスクがCPU資源の割当てを待ち続ける可能性が最も高いものはどれか。

ア.各タスクの優先度を決めて、優先度が高い順に実行し、CPU割当てまでの待ち時間の長さに応じて優先度を徐々に上げていく。

→誤り。動的に優先度を制御することで、各タスクにプロセッサが割り当たりやすくなります

イ.各タスクを実行可能待ち行列に置かれた順に実行し、一定時間が経過したら実行を中断して実行可能待ち行列の最後尾に加える。

→誤り。一定時間が経過すると、処理を中断してプロセッサが別タスクに割り当てられるので、時間が経つとどのタスクもプロセッサが割り当たります

ウ.処理予定時間が最も短いタスクから順に処理を実行する。現在実行中の処理が終了するか、又は何らかの要因によって中断されたとき、次のタスクを開始する。

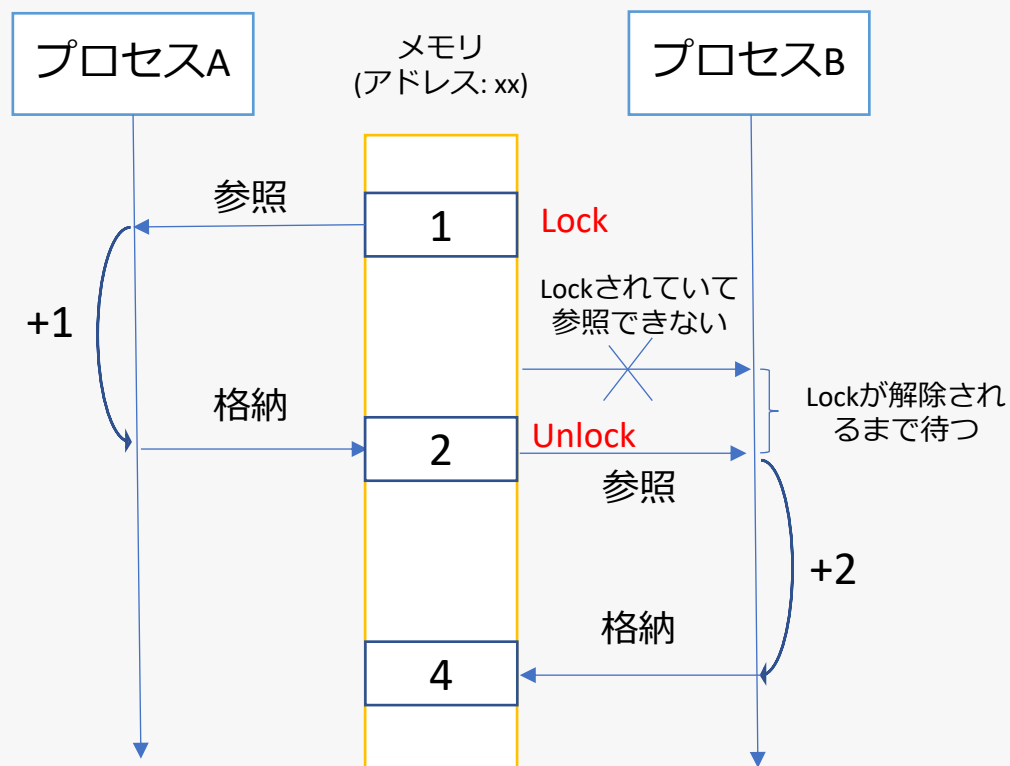
→正しい。処理時間の短いプロセスに割り込まれて、処理時間の長いタスクが待つ可能性が高いです

エ.タスクがシステムに到着した順に実行可能待ち行列の最後尾に加え、常に実行可能待ち行列の先頭のタスクにCPUを割り当てる。

→誤り。到着順には処理されるので一定時間経過すると全タスク処理されます

# プロセスの排他制御

複数のプロセスが同時並行に実行される場合に、資源（メモリ等）の競合を解消するための制御。一つのプロセスがアクセスしている際に別のプロセスのアクセスをさせないようにする



プロセスAが参照している間は、メモリのアドレスにロックをかけてた他のプロセス(プロセスB)がアクセスできないようにする。

## 共有ロック

- ・ 自プロセスと他プロセスが参照することはできるが書き込むことができないロックのこと
- ・ 共有ロックの後に、別のプロセスが共有ロックをかけることができる
- ・ 共有ロックに対して、占有ロックをかけることはできない

## 占有ロック

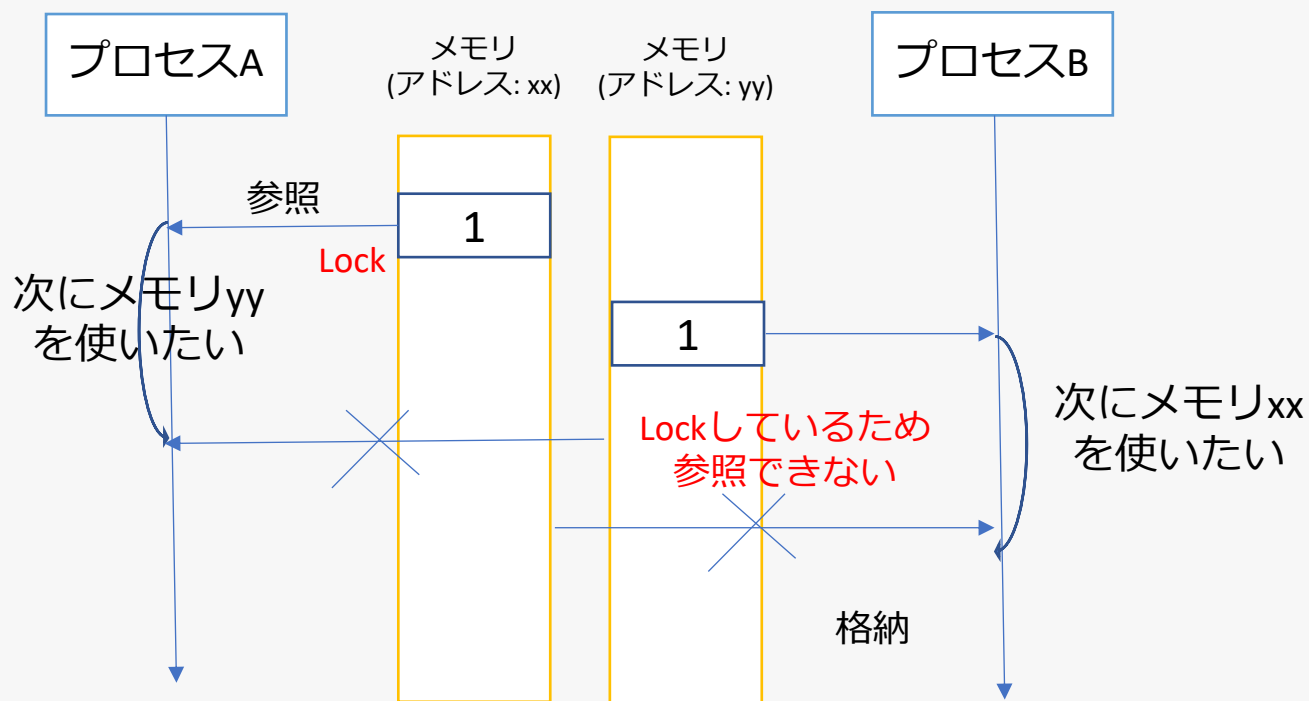
- ・ 自プロセスだけが参照と書き込みができるようにするロックのこと
- ・ 占有ロックの後に、別のプロセスが共有ロック、占有ロックをかけることはできない

## クリティカルセクション

他の処理がアクセスすると処理ができないため、ロックをかけなければいけない部分。

# デッドロック

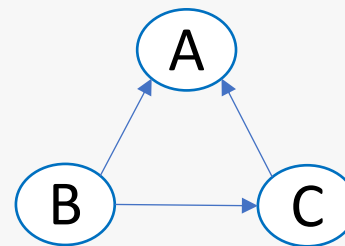
複数のプロセスが互いにロックしたリソースを待つことになり、処理が進まなくなる状態。ロックを検知した場合には、原因となっているプロセスを強制終了する。防ぐには、各プロセスがロックするリソースの順序を添えるなどの工夫が必要



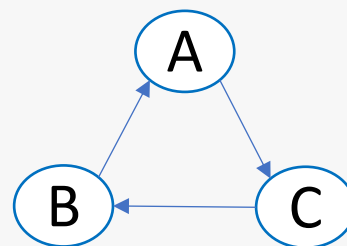
プロセスA、プロセスBはそれぞれがロックしているプロセスを参照しているため、処理ができストップしている

## 待ちグラフ

各プロセスが矢印で指しているプロセスを待っている状態。デッドロックしているプロセスを発見する



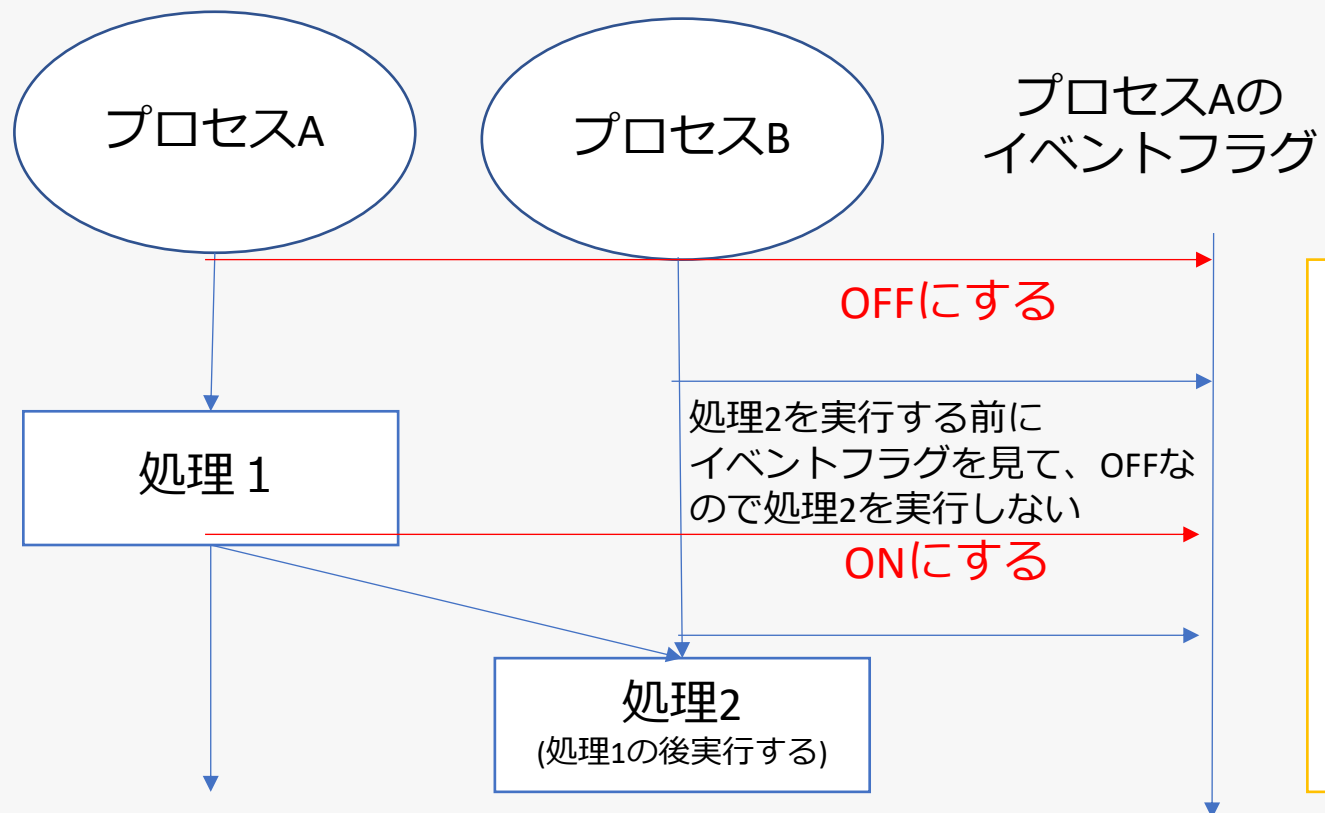
- B, CはAのプロセスを待っている
  - Aが終了すると、Cが実行される
  - Cが終了すると、Bが実行される
- (デッドロックでない)



- AはCのプロセスを待っている
  - CはBのプロセスを待っている
  - BはAのプロセスを待っている
  - 結果どの処理も実行できない
- (デッドロック)

# 同期制御

複数のプロセスが動作している環境では、各プロセス間で同期を取って処理すること  
**イベントフラグ方式**: 各プロセスの状態を表すビット列（イベントフラグ）を持ちます。



- ・あらかじめイベントフラグをOFFに設定します。
- ・処理 1 が終了するとプロセスAがイベントフラグをONにします。
- ・プロセスBはイベントフラグを確認して、ONの場合に、処理 2 を実行します。
- ・結果、処理 1 =>処理2が実行されます

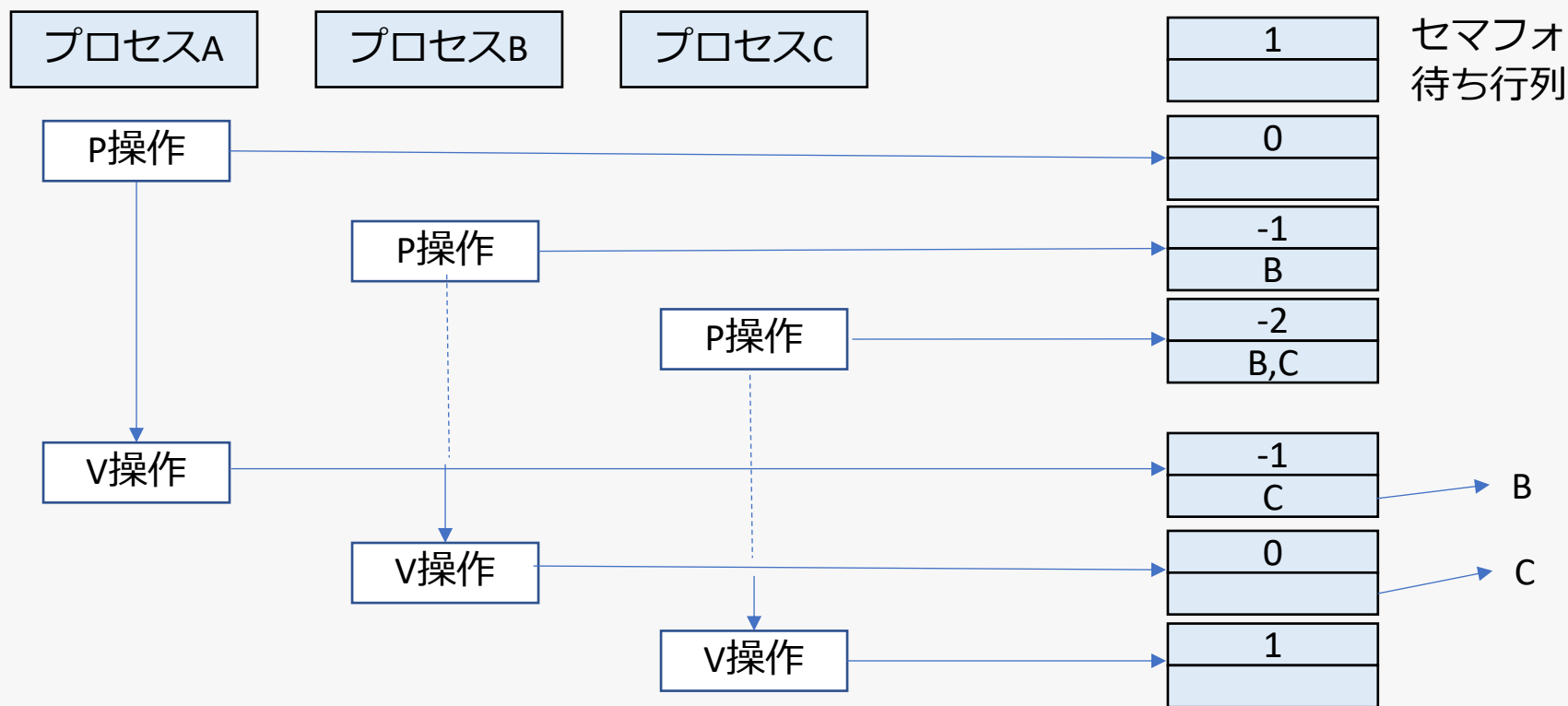
# セマフォシステム

Dijkstraによって考案された同期・排他制御方式

**セマフォ**・・・資源の残量を表す変数

**P操作**・・・クリティカルセクションに入る前に行う。セマフォから1を引く。セマフォの値が負になった場合、プロセスを待ち行列に入れる。

**V操作**・・・クリティカルセクションから出る際に行う。セマフォに1を加える。セマフォの増加前の値が負の場合、待ち行列の先頭プロセスを待ち行列から外して、実行可能状態にする



応用情報技術者平成23年秋期 午前問20

三つの資源X～Zを占有して処理を行う四つのプロセスA～Dがある。各プロセスは処理の進行に伴い、表中の数値の順に資源を占有し、実行終了時に三つの資源を一括して解放する。プロセスAとデッドロックを起こす可能性のあるプロセスはどれか。

| プロセス | 資源の占有順序 |     |     |
|------|---------|-----|-----|
|      | 資源X     | 資源Y | 資源Z |
| A    | 1       | 2   | 3   |
| B    | 1       | 2   | 3   |
| C    | 2       | 3   | 1   |
| D    | 3       | 2   | 1   |

→答えはイ. C, D

Aの順番と逆に資源を占有するプロセスがデッドロックを起こす可能性がある。

A => X, Y, Z   C => Z, X, Y の場合、AがX,Y占有して、次にZを占有しようとしたところ、CがZを占有しているためデッドロックする可能性がある

A => X, Y, Z   D => Z, Y, X の場合、AがXを占有して、次にYを占有しようとしたところ、DがZ, Yを占有しているためデッドロックする可能性がある

- ア. B, C, D
- イ. C, D
- ウ. C
- エ. D

応用情報技術者平成30年春期 午前問18

セマフォを用いる目的として、適切なものはどれか。

ア. 共有資源を管理する。

→正しい。セマフォはプロセスの実行順を制御して、プロセス間で共有する資源へのアクセスとロックを制御します

イ. スタックを容易に実現する。

→誤り。スタックは後入れ先出のメモリです。セマフォとは無関係です

ウ. スラッシングの発生を回避する。

→誤り。スラッシングはページフォルトが多発することによるシステムの処理効率の低下です。セマフォとは無関係です

エ. セグメンテーションを実現する。

→誤り。セグメンテーションとは、セグメントと呼ばれる領域にメモリを分割することです。セマフォとは無関係です

# API(Application Programming Interface)

あるソフトウェアの一部機能を外部に公開して、他のソフトウェアと共有して利用できるようにすること。ソフトウェアの中身を意識せずに他のソフトウェアの機能を利用できる。  
アプリケーションからOSの各機能を利用する。インターネットを通じて外部に公開されている他のシステムの機能を利用する（**Web API**）

## コンポーネントウェア

オブジェクト指向を利用して、標準仕様に基づいた再利用可能なソフトウェア（コンポーネント）を組み合わせてアプリケーションを開発する手法

|            |                                                                                                        |
|------------|--------------------------------------------------------------------------------------------------------|
| Java Beans | Java言語で開発されたコンポーネントを組み合わせて、アプリケーションを開発する。コンポーネントを再利用することができる                                           |
| OSGi       | Javaモジュールを動的に追加・削除・更新できるフレームワーク。コンポーネント指向開発に適しており、モジュール性、SOA、高い拡張性などを実現する。柔軟で拡張性の高いアプリケーション開発を可能にする技術。 |
| CORBA      | オブジェクト指向技術標準化団体（OMB）が制定した分散オブジェクト技術の国際標準仕様のこと。オブジェクト同士がメッセージを交換することができる                                |

# 開発フレームワーク

アプリケーションソフトウェアの標準構造の実装に使われるクラスやライブラリの集まり。主に、Webシステムの開発に用いられる。簡易な開発、セキュリティの向上、性能の向上が見込まれる

|                      |                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------|
| <b>Apache Struts</b> | Java Web Applicationのフレームワーク。クライアントサイド（HTML等）の実装を動的に行う <b>JSP</b> とサーバサイドの実装を行う <b>サーブレット</b> により開発を行う |
| <b>CakePHP</b>       | PHPを用いたフレームワーク。高速なWebアプリケーションの開発を行える。かつてはよく利用されていたが、今はLaravelが主流                                       |
| <b>Spring</b>        | Javaのアプリケーションフレームワーク。保守性が高くて大規模な開発に向いている                                                               |
| <b>Ruby on Rails</b> | Rubyを用いたアプリケーションフレームワーク。簡単に高速で実装できるのが特徴                                                                |

**Hadoop**・・・ペタバイト級の大容量のデータを分散で処理するソフトウェア。MapReduceという分散処理のアルゴリズムに基づいてJavaで実装されている

### 基本情報技術者平成20年春期 午前問29

OSにおけるAPI(Application Program Interface)の説明として、適切なものはどれか。

ア.アプリケーションがハードウェアを直接操作して、各種機能を実現するための仕組みである。

→誤り。デバイスドライバの説明です

イ.アプリケーションから、OSが用意する各種機能を利用するための仕組みである。

→正しい。OSのAPIを利用してOSの機能を利用する

ウ.複数のアプリケーション間でネットワークを介して通信する仕組みである

→誤り。プロセス間通信の説明です

エ.利用者の利便性を図るために、各アプリケーションのメニュー項目を統一する仕組みである。

→誤り。CUA(Common User Access)の説明です

### 基本情報技術者平成20年秋期 午前問68

CORBAの説明はどれか。

ア. Javaを用いた開発において、部品化されたプログラムを作成するための技術仕様である。

→誤り。JavaBeansです

イ.エンドユーザがデータベースにアクセスして、問題点や解決策を得るオンライン分析型アプリケーション機能である。

→誤り。データマートです

ウ.グループでの作業を支援するための電子メール、電子掲示板、ワークフロー機能などから成る統合ソフトである。

→誤り。グループウェアです

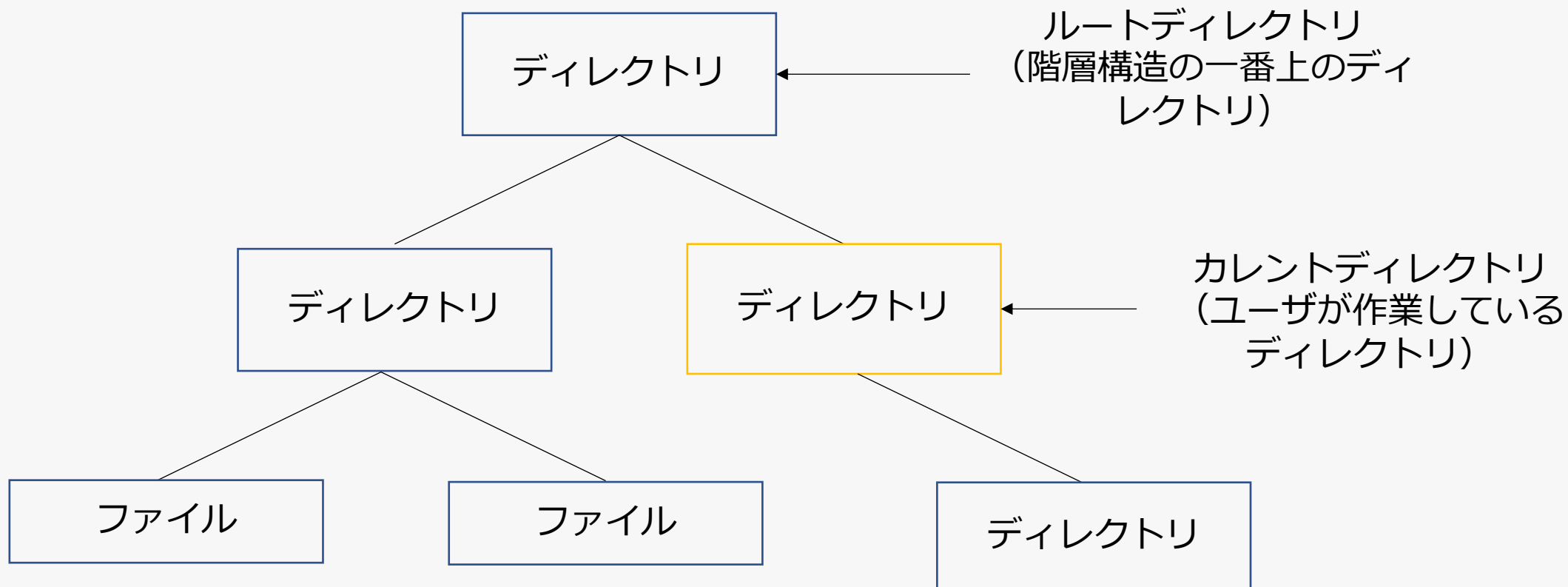
エ.分散システム環境で、オブジェクト同士がメッセージを交換するための共通仕様である。

→正しい。CORBAです

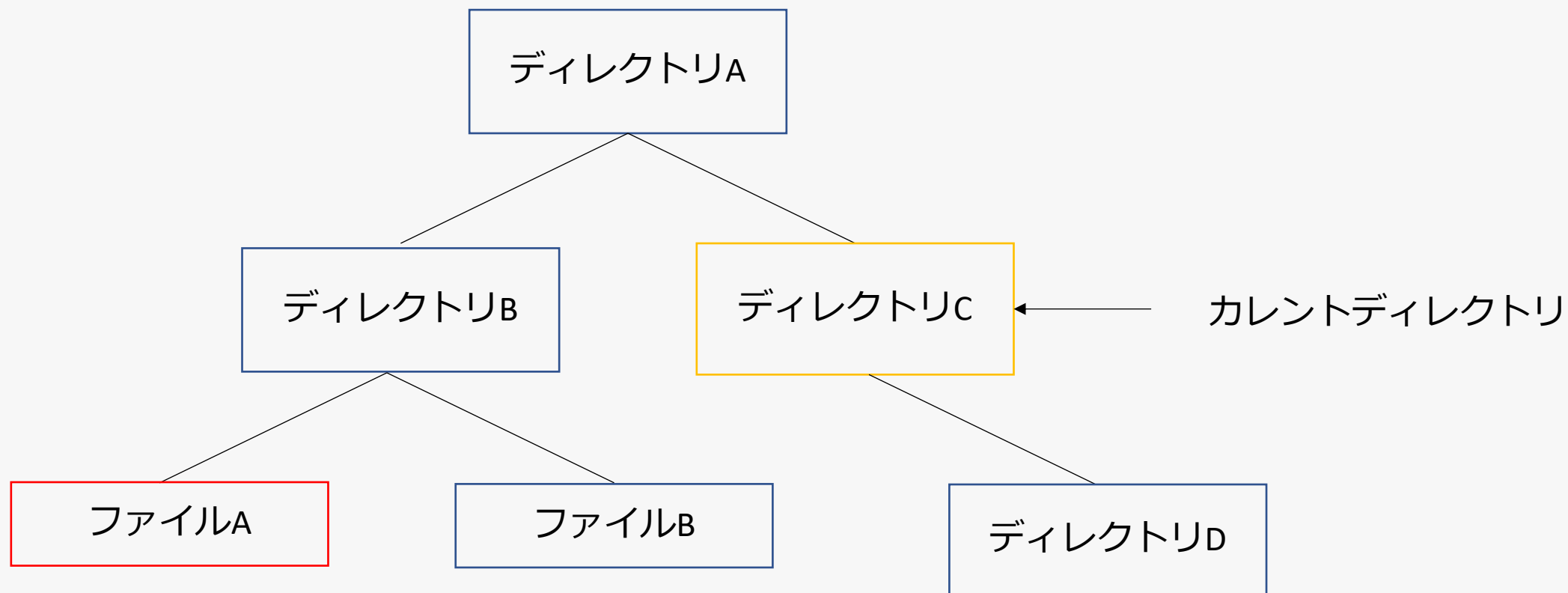
# ファイルとフォルダ、ディレクトリ

パソコン上で保存されるデータは**ファイル**として保存される。ファイル名とファイルの実体を対応付けているものを**フォルダ、ディレクトリ**と言う（Windows, Macの場合フォルダ、Linuxの場合ディレクトリと呼ばれることが多い）

## ディレクトリの階層構造



# 絶対パスと相対パス

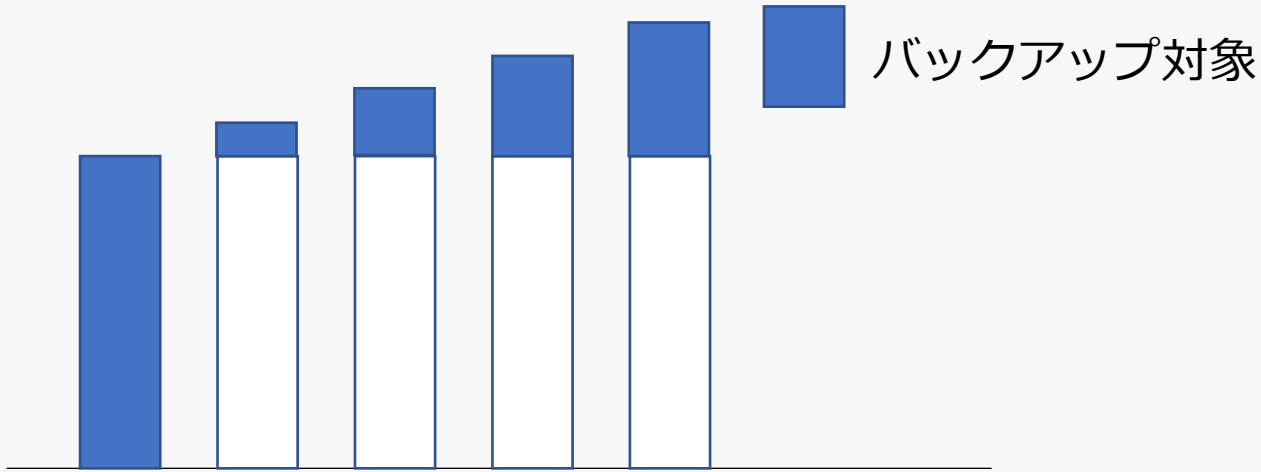


ディレクトリCからファイルAへの絶対パス: /ディレクトリA/ディレクトリB/ファイルA

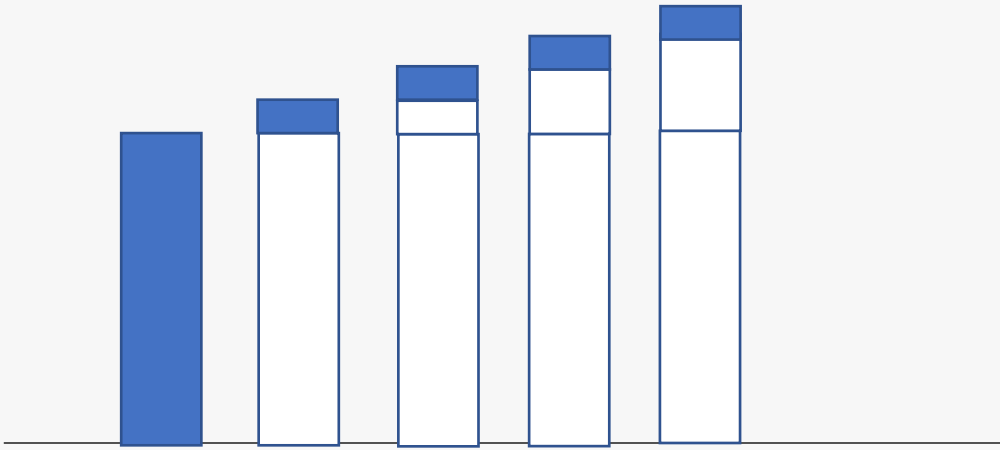
ディレクトリCからファイルAへの相対パス: ../ディレクトリB/ファイルA

# バックアップの方式

|          |                                                                         |
|----------|-------------------------------------------------------------------------|
| フルバックアップ | 毎回、すべてのファイルのバックアップを取る方式。バックアップに要する時間は長くなり、空の状態から元に戻すのに時間は短い             |
| 差分バックアップ | 定期的にフルバックアップを行い、次のフルバックアップまでの期間のバックアップは、フルバック以降に変更のあったすべてのファイルを記録する方式   |
| 増分バックアップ | 定期的にフルバックアップを行い、次のフルバックアップまでの期間のバックアップは、前回の増分バックアップ以降に変更にあったファイルを記録する方式 |



差分バックアップ



増分バックアップ

# ファイルシステム

データをファイルやディレクトリとして編成し、ストレージデバイス上に保存するための方法を定義したもの。異なるオペレーティングシステムや用途に適した機能や性能を提供するために、様々なファイルシステムが開発されている。



|      |                                                                                                                                                               |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FAT  | <b>古くからあるファイルシステム</b> で、MS-DOSやWindows 9x系で使用されていた。 <b>シンプルな構造</b> で、小容量のストレージに適している。現在でも、USBメモリやSDカードなどのリムーバブルメディアで広く使用されている                                 |
| NTFS | FATと比べて、大容量のストレージや大きなファイルをサポートし、 <b>セキュリティ機能やジャーナリング機能</b> を備えている。現在の <b>Windows OS</b> で広く使用されている。                                                           |
| NFS  | <b>Unix系オペレーティングシステム</b> で使用する分散ファイルシステム。ネットワーク上の複数のコンピュータ間で <b>ファイルを共有することができる</b> 。NFSを使用することで、リモートのファイルシステムをローカルのファイルシステムのようにマウントして使用できる。                  |
| ReFS | Microsoftが開発した新しいファイルシステムで、Windows Server 2012で導入された。 <b>大容量のストレージと高い信頼性を必要とするシステム</b> に適している。データの完全性を保護し、 <b>オンラインの修復機能</b> を備えている。主にサーバー環境で使用されることを目的としている。 |

**基本情報技術者平成26年秋期 午前問19**

ファイルシステムの絶対パス名を説明したものはどれか。

ア.あるディレクトリから対象ファイルに至る幾つかのパス名のうち、最短のパス名

→誤り。

イ.カレントディレクトリから対象ファイルに至るパス名

→誤り。相対パスです

ウ.ホームディレクトリから対象ファイルに至るパス名

→誤り。

エ.ルートディレクトリから対象ファイルに至るパス名

→正しい。ルートディレクトリからのパスが絶対パスです

**基本情報技術者令和元年秋期 午前問19**

バックアップ方式の説明のうち、増分バックアップはどれか。ここで、最初のバックアップでは、全てのファイルのバックアップを取得し、OSが管理しているファイル更新を示す情報はリセットされるものとする。

ア.最初のバックアップの後、ファイル更新を示す情報があるファイルだけをバックアップし、ファイル更新を示す情報は変更しないでそのまま残しておく。

→誤り。差分バックアップの説明です

イ.最初のバックアップの後、ファイル更新を示す情報にかかわらず、全てのファイルをバックアップし、ファイル更新を示す情報はリセットする。

→誤り。フルバックアップの説明です

ウ.直前に行ったバックアップの後、ファイル更新を示す情報があるファイルだけをバックアップし、ファイル更新を示す情報はリセットする。

→正しい。増分バックアップの説明です

エ.直前に行ったバックアップの後、ファイル更新を示す情報にかかわらず、全てのファイルをバックアップし、ファイル更新を示す情報は変更しないでそのまま残しておく。

→誤り。フルバックアップの説明です

## まとめ

**OSの種類:** Windows、MacOS、Unix、Linux(Debian, Redhat)、Android, ios

**OSの起動:** BIOS, マルチブート

**OSの役割:** ユーザの管理、ハードウェア資源の管理、タスクの管理、ファイルの管理、周辺機器の管理

**メモリ管理:** 断片化(フラグメンテーション)、コンパクション

**空きメモリ確保のアルゴリズム:** ファーストフィット、ベストフィット、オーバレイ

**仮想メモリ:** ページング、ページフォールト、ページイン、ページアウト、ページリプレースメント、デマンドページング、プリページング

**ページ置換えアルゴリズム:** FIFO, LIFO, LRU, LFU

スラッシング

**プロセスの状態遷移:** 実行状態、実行可能状態、待ち状態、ディスパッチ、コンテキスト切替え(プリエンプティブ、ノンプリエンプティブ)

**スケジューリング方式:** ランドロビン方式、優先度順方式、到着順方式、SJF方式、多重待ち行列方式、フィードバック待ち行列方式

**プロセスの排他制御:** 共有ロック、占有ロック、クリティカルセクション、デッドロック、同期制御、イベントフラグ方式、セマフォ方式  
API

**コンポーネントウェア:** Java Beans, ActiveX, CORBA

**開発フレームワーク:** Apache Struts, CakePHP, Spring, Ruby on Rails

Hadoop

ファイル、フォルダ、ディレクトリの階層構造

絶対パス、相対パス

**バックアップの方式:** フルバックアップ、差分バックアップ、増分バックアップ

**ファイルシステム:** FAT、NTFS、NFS、ReFS

過去問（オペレーティングシステム、ミ  
ドルウェア、ファイルシステム）

## 「出典：応用情報技術者令和4年秋期 午前問17」

ほとんどのプログラムの大きさがページサイズの半分以下のシステムにおいて、ページサイズを半分にしたときに予想されるものはどれか。ここで、このシステムは主記憶が不足しがちで、多重度やスループットなどはシステム性能の限界で運用しているものとする。

ア. ページサイズが小さくなるので、領域管理などのオーバーヘッドが減少する。

イ. ページ内に余裕がなくなるので、ページ置換えによってシステム性能が低下する。

ウ. ページ内の無駄な空き領域が減少するので、主記憶不足が緩和される。

エ. ページフォールトの回数が増加するので、システム性能が低下する。

## 「出典：応用情報技術者令和4年秋期 午前問17」

ほとんどのプログラムの大きさがページサイズの半分以下のシステムにおいて、ページサイズを半分にしたときに予想されるものはどれか。ここで、このシステムは主記憶が不足しがちで、多重度やスループットなどはシステム性能の限界で運用しているものとする。

ア. ページサイズが小さくなるので、領域管理などのオーバーヘッドが減少する。

→誤り。ページサイズを小さくすると、ページテーブルのエントリ数が増加します。ページテーブルは、仮想アドレスと物理アドレスの変換テーブルであり、エントリ数が増えると管理にオーバーヘッドがかかります。

イ. ページ内に余裕がなくなるので、ページ置換えによってシステム性能が低下する。

→誤り。ほとんどのプログラムがページサイズの半分以下であるとされています。そのため、ページサイズを半分にしても、大部分のプログラムは1つのページ内に収まります。

ウ. ページ内の無駄な空き領域が減少するので、主記憶不足が緩和される。

→正しい。主記憶が不足しがちなシステムでは、この空き領域の減少が主記憶不足の緩和につながります。

エ. ページフォールトの回数が増加するので、システム性能が低下する。

→誤り。ほとんどのプログラムがページサイズの半分以下であるとされています。そのため、ページフォールトの発生確率は増加しません。

# 「出典：応用情報技術者令和元年秋期 午前問18」

ページング方式の仮想記憶において、主記憶に存在しないページをアクセスした場合の処理や状態の順番として、適切なものはどれか。ここで、現在主記憶には、空きページはないものとする。

ア.置換え対象ページの決定→ページイン→ページフォールト→ページアウト

イ.置換え対象ページの決定→ページフォールト→ページアウト→ページイン

ウ.ページフォールト→置換え対象ページの決定→ページアウト→ページイン

エ.ページフォールト→置換え対象ページの決定→ページイン→ページアウト

# 「出典：応用情報技術者令和元年秋期 午前問18」

ページング方式の仮想記憶において、主記憶に存在しないページをアクセスした場合の処理や状態の順番として、適切なものはどれか。ここで、現在主記憶には、空きページはないものとする。

ア.置換え対象ページの決定→ページイン→ページフォールト→ページアウト

イ.置換え対象ページの決定→ページフォールト→ページアウト→ページイン

ウ.ページフォールト→置換え対象ページの決定→ページアウト→ページイン

エ.ページフォールト→置換え対象ページの決定→ページイン→ページアウト

# 「出典：応用情報技術者令和元年秋期 午前問18」

ページング方式の仮想記憶において、主記憶に存在しないページをアクセスした場合の処理や状態の順番として、適切なものはどれか。ここで、現在主記憶には、空きページはないものとする。

ア.置換え対象ページの決定→ページイン→ページフォールト→ページアウト

イ.置換え対象ページの決定→ページフォールト→ページアウト→ページイン

**ウ.ページフォールト→置換え対象ページの決定→ページアウト→ページイン**

ページング処理の実行順序としては、1.必要なデータが主記憶上に存在しない状態の発生（ページフォールト） 2. 置き換え対象のページをFIFO,LRU,LFUなどのアルゴリズムで決定 3. 置き換え対象のページを主記憶から仮想記憶に退避（ページアウト）。4.必要なページを仮想記憶から主記憶に移行（ページイン）

エ.ページフォールト→置換え対象ページの決定→ページイン→ページアウト

## 「出典：応用情報技術者平成31年春期 午前問18」

二つのタスクが共用する二つの資源を排他的に使用するとき、デッドロックが発生するおそれがある。このデッドロックの発生を防ぐ方法はどれか。

ア.一方のタスクの優先度を高くする。

イ.資源獲得の順序を両方のタスクで同じにする。

ウ.資源獲得の順序を両方のタスクで逆にする。

エ.両方のタスクの優先度を同じにする。

# 「出典：応用情報技術者平成31年春期 午前問18」

二つのタスクが共用する二つの資源を排他的に使用するとき、デッドロックが発生するおそれがある。このデッドロックの発生を防ぐ方法はどれか。

ア.一方のタスクの優先度を高くする。

→デッドロックの発生と優先度は無関係です

**イ.資源獲得の順序を両方のタスクで同じにする。**

→正しい。資源獲得の順序を同じにするとデッドロックを防ぐことができます

ウ.資源獲得の順序を両方のタスクで逆にする。

→デッドロックは、資源獲得順が逆の場合に発生します。

エ.両方のタスクの優先度を同じにする。

→デッドロックの発生と優先度は無関係です

## 「出典：基本情報技術者令和元年秋期 午前問18」

優先度に基づくプリエンプティブなスケジューリングを行うリアルタイムOSで、二つのタスクA、Bをスケジューリングする。Aの方がBより優先度が高い場合にリアルタイムOSが行う動作のうち、適切なものはどれか。

ア. Aの実行中にBに起動がかかると、Aを実行可能状態にしてBを実行する。

イ. Aの実行中にBに起動がかかると、Aを待ち状態にしてBを実行する。

ウ. Bの実行中にAに起動がかかると、Bを実行可能状態にしてAを実行する。

エ. Bの実行中にAに起動がかかると、Bを待ち状態にしてAを実行する。

# 「出典：基本情報技術者令和元年秋期 午前問18」

優先度に基づくプリエンプティブなスケジューリングを行うリアルタイムOSで、二つのタスクA, Bをスケジューリングする。Aの方がBより優先度が高い場合にリアルタイムOSが行う動作のうち、適切なものはどれか。

ア. Aの実行中にBに起動がかかると、Aを実行可能状態にしてBを実行する。  
→Aの方が優先度が高いため、タスクは切り替わらないです。

イ. Aの実行中にBに起動がかかると、Aを待ち状態にしてBを実行する。  
→Aの方が優先度が高いため、タスクは切り替わらないです。

**ウ. Bの実行中にAに起動がかかると、Bを実行可能状態にしてAを実行する。**  
→Aの方が優先度が高いため、タスクは切り替わります。プリエンプティブなスケジューリングのため、Bが実行可能状態にAが実行中に移行します

エ. Bの実行中にAに起動がかかると、Bを待ち状態にしてAを実行する。  
→誤り。Bは実行可能状態に移行します。

# テクノロジー系試験範囲

## 1. 基礎理論

### 1-1. 基礎理論

離散数学/応用数学/情報に関する理論/通信に関する理論/継続・制御に関する理論

### 1-2. アルゴリズムとプログラミング

データ構造/アルゴリズム/プログラミング/プログラム言語/その他の言語

## 2. コンピュータシステム

### 2-1. コンピュータ構成要素

プロセッサ/メモリ/バス/入出力デバイス/入出力装置

### 2-2. システム構成要素

システムの構成/システムの評価指標

### 2-3. ソフトウェア

オペレーティングシステム/ミドルウェア/ファイルシステム/開発ツール/オープンソースソフトウェア

### 2-4. ハードウェア

ハードウェア

# テクノロジー系試験範囲

## 3. 技術要素

### 3-1. ヒューマンインターフェース

ヒューマンインターフェース技術/インタフェース設計

### 3-2. マルチメディア

マルチメディア技術/マルチメディア応用

### 3-3. データベース

データベース方式/データベース設計/データ操作/トランザクション処理/データベース応用

### 3-4. ネットワーク

ネットワーク方式/データ通信と制御/通信プロトコル/ネットワーク管理/ネットワーク応用

### 3-5. セキュリティ

情報セキュリティ/情報セキュリティ管理/セキュリティ技術評価/情報セキュリティ対策/情報セキュリティ実装技術

## 4. 開発技術

### 4-1. システム開発技術

システム要件定義/システム方式設計/ソフトウェア要件定義/ソフトウェア方式設計・ソフトウェア詳細設計/ソフトウェア構築/ソフトウェア結合・ソフトウェア適格性確認テスト/ソフトウェア結合・ソフトウェア適格性確認テスト/システム結合・システム適格性確認テスト/導入/受入れ支援/保守・廃棄

### 4-2. ソフトウェア開発管理技術

開発プロセス・手法/知的財産権運用管理/開発環境管理/構成管理・変更管理

# コンピュータシステム

## ソフトウェア

## 開発ツール、オープンソースソフトウェア

- 目標：**
- ソフトウェア開発に用いる開発ツールの種類, 特徴, 機能を修得する。
  - オープンソースソフトウェアの種類, 特徴, 機能, 構成, 利用上の考慮点, 動向を修得する。

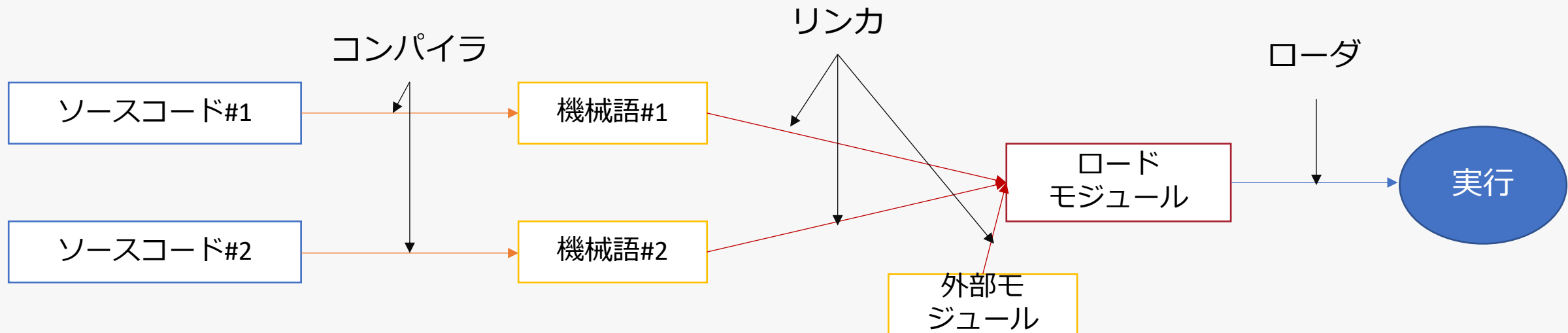
# 開発ツール

設計を支援するツール，プログラミングやテストを支援するツール，ソフトウェア開発の各工程を通して自動化，効率化を目的とするツール，開発作業全体を一貫して工程を支援する IDE（Integrated Development Environment：統合開発環境）などシステムの開発に利用するツール

|            |                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------|
| パーサ        | ソースコードの構文解析を行うツール。プログラムやXMLの文書が言語の構文規則に従って、記述されているのかチェックする                                               |
| コードオーディタ   | 作成したコードが基準や規約に合致しているのか確認するツール                                                                            |
| トレース       | プログラムが実行した経路を追跡・確認するツール                                                                                  |
| インスペクタ     | プログラム実行時にデータの内容を表示するツール。デバッグ時に用いられる                                                                      |
| プロファイラ     | 性能解析ツール。プログラム実行時にどの関数が何回実行されて、どれくらいの時間を消費したのかを確認するのに用いられる                                                |
| アサーションチェッカ | プログラム実行中の特定の時点で成立する変数間の関係や条件を記述した論理式を埋め込んで、そのプログラムの正当性を検証すること                                            |
| シミュレータ     | 仮想的なモデルをPC上に作成して、模擬的に試験をするソフトウェア                                                                         |
| 静的コード解析    | プログラム実行しない状態でプログラムをチェックするツール。コーディングルールに準拠しているのか、インデント等書式に問題がないのか、命名規約等の違反はないのか確認する                       |
| テストカバレッジ分析 | テストの網羅率(カバレッジ)を定量的に測定する。プログラム実行時にプログラムのどの部分が実行されてどの部分が実行されていないのかを見てテストの品質を確認する。そのためのツールは <b>カバレッジモニタ</b> |
| 静的テストツール   | プログラムを実行しない状態で、ソースコードを解析してプログラムの誤りを検出する（パーサ、コードオーディタ）                                                    |
| 動的テストツール   | プログラムを実行した状態で、プログラムの誤りを検出する（インスペクタ、トレース、プロファイラ、アサーションチェッカ、カバレッジモニタ）                                      |

# 言語処理ツール

|          |                                                |
|----------|------------------------------------------------|
| コンパイラ    | ソースコードをアセンブラ言語や機械語に翻訳をする                       |
| リンカ      | コンパイルされたモジュールと外部のモジュール同士を結合して、1つのロードモジュールを作成する |
| ローダ      | 実行に先立ってロードモジュールを主記憶にロードする                      |
| インタプリタ   | 高水準の言語で書かれたプログラムを1行ずつ解釈して実行する                  |
| クロスコンパイラ | あるコンピュータ上で、異なる命令形式の別のコンピュータで動作できるプログラムを作成すること  |
| コンバータ    | ある言語で記載されたソースコードを別の言語のソースコードに変換するプログラム         |



**応用情報技術者平成30年秋期 午前問19**

プログラムの性能を改善するに当たって、関数、文などの実行回数や実行時間を計測して統計を取るために用いるツールはどれか。

ア.コンパイラ

→誤り。ソースコードをアセンブラ言語や機械語に翻訳をするツール

イ.デバッガ

→誤り。プログラムを一文一文実行して一時停止してその時の変数の状態を確認し、バグの解析を行うツール

ウ.パーサ

→誤り。ソースコードの構文解析を行うツールです

エ.プロファイラ

→正しい。プロファイラは関数の実行回数、実行時間を計測する性能解析ツールです

**基本情報技術者平成28年春期 午前問20**

リンカの機能として、適切なものはどれか。

ア.作成したプログラムをライブラリに登録する。

→誤り。これはコンパイラの機能です

イ.実行に先立ってロードモジュールを主記憶にロードする。

→誤り。これはローダの機能です

ウ.相互参照の解決などを行い、複数の目的モジュールなどから一つのロードモジュールを生成する。

→正しい。

エ.プログラムの実行を監視し、ステップごとに実行結果を記録する。

→誤り。これはデバッガの機能です

# オープンソースソフトウェア(OSS)

より良いソフトウェアにするために、ソースコードを公開し、再配布を自由にできるソフトウェア。OSSを採用することで費用を削減できるが、技術サポートを受けられない、保証がないなどの短所もある

## オープンソースソフトウェア(OSS)の例

|              |                           |
|--------------|---------------------------|
| OS           | Linux, Android            |
| Webブラウザ      | Firefox                   |
| Webサーバ       | Apache                    |
| アプリケーションサーバ  | Tomcat                    |
| データベース(RDBS) | MySQL, PostgreSQL, SQLite |
| オフィス系ソフトウェア  | OpenOffice                |
| メールソフト       | Thunderbird               |
| プロキシサーバ      | Squid                     |
| DNSサーバ       | BIND                      |
| 統合開発環境       | Eclipse                   |

# OSSのライセンスの種類

|                            |                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| GPLライセンス<br>(GNU GPLライセンス) | 無保証、著作権表示の保持、同一ライセンスの適用が条件で、改変、再配布が認められている。GPLライセンスのソフトウェアも改変したものもGPLライセンスで、ソースコードの <b>公開</b> をしなければならない |
| BSDライセンス                   | 無保証、著作権・ライセンス条文の表示を条件に、改変や再配布を認めている。改変した場合のソースコードは <b>非公開</b> でよい                                        |
| LGPLライセンス                  | GPLの公開要件を緩めたライセンス。LGPLライセンスのプログラムを <b>動的リンクするプログラム</b> は、ソースコードを非公開にして配布できる。                             |
| MPLライセンス                   | LGPLの公開要件を緩めたもの。MPLライセンスのプログラムを別ファイルとして利用するようなプログラム（ <b>静的にリンクするプログラム</b> ）は、非公開のまま再配布できる                |
| Apacheライセンス                | <b>Apacheソフトウェア財団</b> が定めたライセンス。再配布に当たってはApacheのコードを利用している事の明示、改変した場合には、その旨を告知することが必要                    |
| デュアルライセンス                  | 1つのソフトウェアを2つ以上のライセンスの元で配布する。                                                                             |

**応用情報技術者平成29年春期 午前問19**

オープンソースライセンスのGNU GPL(GNU General Public License)の説明のうち、適切なものはどれか。

ア. GPLであるソースコードの実現する機能を利用する、別のプログラムのソースコードを作成すると、GPLが適用される。

→誤り。別のプログラムがGPLライセンスでない場合には、GPLは適用されません

イ. GPLであるソースコードの全てを使って派生物を作った場合に限り、GPLが適用される。

→誤り。GPLのソースコードを一部でも利用した派生物はGPLです

ウ. GPLであるソースコードの派生物のライセンスは、無条件にGPLとなる。

→正しい。GPLのソフトウェアの派生物もGPLライセンスになります

エ. GPLであるソースコードを組み込んだ派生物をGPLにするか否かは、派生物の開発者が決める。

→誤り。開発者の意向でなく、GPLのソースコードを組み込むとGPLライセンスになります

**基本情報技術者平成27年秋期 午前問20**

オープンソースの統合開発環境であって、アプリケーション開発のためのソフトウェア及び支援ツール類をまとめたものはどれか。

ア. Eclipse

→正しい。Eclipseがオープンソースの統合開発環境です

イ. Perl

→誤り。Perlはスクリプト言語の一種です

ウ. PHP

→誤り。PHPはスクリプト言語の一種です

エ. Ruby

→誤り。Rubyはスクリプト言語の一種で日本人が開発したものです

# 仮想化・クラウド系OSS



|            |                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------|
| Xen        | 仮想マシン（VM）を作成・管理するために使用されるハイパーバイザー。複数のゲストオペレーティングシステムを単一のハードウェア上で実行できる。                                                       |
| Docker     | コンテナ化技術で、アプリケーションとその依存関係をパッケージ化し、移植性と拡張性を向上させる。開発、テスト、デプロイを簡素化し、マイクロサービスアーキテクチャを実現するために広く使用されている                             |
| Kubernetes | アプリケーションを動かすコンテナを管理・自動制御するツール。負荷に応じてコンテナの数を増減し、障害時には自動で再起動するなど、大規模アプリケーションの運用が容易になる。                                         |
| Terraform  | クラウド上のサーバーやネットワークなどのインフラをコードで自動構築するツール。同じ環境を簡単に再現でき、人的ミスも減らせる。主要クラウドに対応し、インフラ構築が簡単にする。                                       |
| OpenStack  | クラウドを作成するためのソフトウェア。ストレージ、ネットワークなどのリソースを管理し、プライベートクラウドやパブリッククラウドの構築に使用される。                                                    |
| Istio      | マイクロサービスアーキテクチャで構築されたアプリケーションの通信を管理・制御するツール。サービス間の通信を管理し、トラフィックルーティング、セキュリティ、観測性、ポリシー適用などの機能を提供する。Kubernetesと連携して使用されることが多い。 |



|                |                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Chainer        | Pythonベースのディープラーニングフレームワークで、 <b>柔軟性と使いやすさを重視している</b> 。 動的な計算グラフを使用し、ニューラルネットの定義と訓練を簡単に行える。                                          |
| OpenCV         | <b>コンピュータビジョン</b> とマシンラーニングのためのオープンソースライブラリで、C++、Python、Javaなどの言語をサポート。 OpenCVは、画像処理、物体検出、顔認識などの機能を提供し、幅広い分野で使用されている。               |
| PyTorch        | Pythonベースのディープラーニングフレームワークで、 <b>Facebook(Meta)</b> が開発。 Pythonの動的型付けの利点を活かした開発が可能で、行列演算とグラフベースの計算の両方をサポート。                          |
| TensorFlow     | <b>Google</b> が開発したオープンソースのディープラーニングフレームワークで、 <b>大規模な機械学習モデルの構築と訓練に使用される</b> 。 TensorFlowは、静的な計算グラフを使用し、分散処理をサポート。                  |
| Keras          | Pythonで書かれた、Tensorflowを元にしたユーザーフレンドリーなディープラーニングライブラリ。                                                                               |
| Deeplearning4j | <b>Java</b> のスケラブルなディープラーニングライブラリで、分散処理フレームワークであるApache Sparkと連携して動作する。 Deeplearning4jは、企業環境でのディープラーニングの導入を容易にし、JVMエコシステムとの統合を提供する。 |

# システム開発・運用支援OSS



|                  |                                                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------------------------|
| Eclipse          | Javaを中心とした様々なプログラミング言語に対応した <b>統合開発環境(IDE)</b> 。コーディング、デバッグ、テストといった開発作業に必要な機能を一つにまとめ、開発者の生産性を大幅に向上させる。                       |
| Subversion (SVN) | <b>バージョン管理システム</b> で、ファイルの変更履歴を一元的に管理することができる。ただし、近年はより高機能なGitに人気が移っている。                                                     |
| Git              | 分散型バージョン管理システムの1つ、 <b>SVNと比較して高速な動作とオフラインでの作業が可能</b> 。ブランチ機能が強力で、複数の開発ラインを同時に進めるような複雑な開発にも対応できる。                             |
| SonarQube        | <b>コードの品質を管理</b> するためのツールで、コードの複雑度、バグの可能性、脆弱性などを自動的に分析し、視覚的に表示する。開発者はこれらの情報をもとに、コードの品質を改善できる。継続的インテグレーション(CI)との連携も可能。        |
| Jenkins          | 継続的インテグレーション(CI)と継続的デリバリー(CD)を実現するためのツール。コードの変更を自動的に検知し、 <b>ビルド、テスト、デプロイといった一連の工程を自動で実行</b> する。これにより、人的エラーを減らし、開発サイクルを速められる。 |
| Redmine          | <b>プロジェクト管理</b> のためのソフトウェア。チケット管理、Wiki、ガントチャートといった機能を提供し、タスクの進捗状況や関連情報を一元的に管理できる。                                            |
| Selenium         | <b>Webアプリケーションのテストを自動化</b> するためのツール。ブラウザ上の操作を記録し、その操作を再生することで、テストを自動的に実行することができる。                                            |

|                   |                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ansible</b>    | <b>サーバーの構成管理ツール</b> の1つ。YAML形式で記述された <b>手順書(Playbook)</b> に従って、サーバーを自動的に設定する。シンプルな記述で複雑な処理を実現でき、人的エラーを防ぐことができる。                                   |
| <b>Chef</b>       | <b>サーバーの構成管理ツール</b> の1つ。Ruby言語で記述された <b>レシピ(Recipe)</b> を使って、サーバーの設定を宣言的に管理する。レシピを書くことで、サーバーの構成をコードとして管理でき、インフラのコード化を実現する。                        |
| <b>Puppet</b>     | <b>サーバーの構成管理ツール</b> の1つ。宣言的な記述方式を採用しています。Puppetを使うことで、サーバーの設定を一元的に管理することができ、多数のサーバーの構成を統一的に維持することが可能。AnsibleやChefと並んで、広く利用される。                    |
| <b>Prometheus</b> | <b>監視とアラート</b> のためのツールで、サーバーやアプリケーションから情報を収集し、一元的に保存・管理します。リアルタイムな監視とアラート通知を行うことができ、システムの異常をいち早く検知する                                              |
| <b>Zabbix</b>     | <b>監視ツール</b> で、サーバーやネットワーク機器、アプリケーションなどの稼働状況を監視できる。異常を検知した際にはアラートを発報し、迅速な障害対応を支援する。幅広いプラットフォームに対応し、大規模なシステムの監視にも適している。                            |
| <b>Grafana</b>    | <b>データの可視化を行うツール</b> 。Prometheusなどの監視ツールと連携して使われることが多い。収集されたメトリクスデータをグラフや図表で直感的に表示することができ、システムの状態を一目で把握することができる。ダッシュボードを作成し、複数のグラフをまとめて表示することも可能。 |



## まとめ

**開発ツール**・・・パーサ、コードオーディタ、トレーサ、インスペクタ、プロファイラ、アサーションチェッカ、シミュレータ、静的コード解析、テストカバレッジ分析、静的テストツール、動的テストツール

**言語処理ツール**・・・コンパイラ、リンカ、ローダ、インタプリタ、クロスコンパイラ、コンバータ

**オープンソースソフトウェア**・・・

Linux,Android,FireFox,Apache,Tomcat,MySQL,PostgreSQL,SQLite,OpenOffice,Thunderbird,Squid,BIND,Eclipse

**OSSライセンス**・・・GPLライセンス、BSDライセンス、LGPLライセンス、MPLライセンス、Apacheライセンス、デュアルライセンス

**仮想化・クラウド系oss**・・・Xen、Docker、Kubernetes、Terraform、OpenStack、Istio

**AI系OSS**・・・Chainer、OpenCV、PyTorch、TensorFlow、Keras、Deeplearning4j

**システム開発・運用支援oss**・・・Eclipse、Subversion(SVN)、Git、SonarQube、Jenkins、Redmine、Selenium、Ansible、Chef、Puppet、Prometheus、Zabbix、Grafana

過去問（開発ツール、オープンソースソフトウェア）

## 「出典：応用情報技術者令和5年秋期 午前問19」

Linuxカーネルの説明として、適切なものはどれか。

ア. CUIによるコマンド入力のためのシェルと呼ばれるソフトウェアが組み込まれていて、文字での操作が可能である。

イ. GUIを利用できるデスクトップ環境が組み込まれていて、マウスを使った直感的な操作が可能である。

ウ. Webブラウザ，ワープロソフト，表計算ソフトなどが含まれており，Linuxカーネルだけで多くの業務が行える。

エ. プロセス管理やメモリ管理などの，アプリケーションソフトウェアが動作するための基本機能を提供する。

# 「出典：応用情報技術者令和5年秋期 午前問19」

Linuxカーネルの説明として、適切なものはどれか。

ア. CUIによるコマンド入力のためのシェルと呼ばれるソフトウェアが組み込まれていて、文字での操作が可能である。

→誤り。シェルは、Linuxカーネルの上で動作するソフトウェアであり、カーネル自体には組み込まれていません。

イ. GUIを利用できるデスクトップ環境が組み込まれていて、マウスを使った直感的な操作が可能である。

→誤り。デスクトップ環境は、Linuxカーネルの上で動作するソフトウェアです

ウ. Webブラウザ、ワープロソフト、表計算ソフトなどが含まれており、Linuxカーネルだけで多くの業務が行える。

→誤り。Webブラウザ、ワープロソフト、表計算ソフトなどのアプリケーションソフトウェアは、Linuxカーネルの上で動作するものであり、カーネル自体には含まれていません。

**エ. プロセス管理やメモリ管理などの、アプリケーションソフトウェアが動作するための基本機能を提供する。**

→正しい。Linuxカーネルは、オペレーティングシステムの中核であり、プロセス管理、メモリ管理、デバイス管理などの基本的な機能を提供します。アプリケーションソフトウェアは、これらの機能の上で動作します。

## 「出典：応用情報技術者令和5年春期 午前問20」

コンテナ型仮想化の環境であって、アプリケーションソフトウェアの構築、実行、管理を行うためのプラットフォームを提供するOSSはどれか。

ア. Docker

イ. KVM

ウ. QEMU

エ. Xen

## 「出典：応用情報技術者令和5年春期 午前問20」

コンテナ型仮想化の環境であって、アプリケーションソフトウェアの構築、実行、管理を行うためのプラットフォームを提供するOSSはどれか。

### ア. Docker

→正しい。Dockerは、コンテナ型仮想化技術を用いたオープンソースのプラットフォームであり、アプリケーションソフトウェアの構築、実行、管理を容易にします。Dockerは、アプリケーションとその依存関係をコンテナにパッケージ化することで、移植性と拡張性を向上させます。

### イ. KVM

→誤り。コンテナ型仮想化ではなく、仮想マシン（VM）を作成・管理するために使用されます。

### ウ. QEMU

→誤り。仮想マシン（VM）を作成・管理するために使用されますが、コンテナ型仮想化のプラットフォームではありません。

### エ. Xen

→誤り。仮想マシン（VM）を作成・管理するために使用されますが、コンテナ型仮想化のプラットフォームではありません。

# 「出典：応用情報技術者平成27年春期 午前問19」

あるコンピュータ上で、異なる命令形式のコンピュータで実行できる目的プログラムを生成する言語処理プログラムはどれか。

ア.エミュレータ

イ.クロスコンパイラ

ウ.最適化コンパイラ

エ.プログラムジェネレータ

# 「出典：応用情報技術者平成27年春期 午前問19」

あるコンピュータ上で、異なる命令形式のコンピュータで実行できる目的プログラムを生成する言語処理プログラムはどれか。

ア.エミュレータ

→誤り。エミュレータはシステムの挙動を模倣して再現するソフトウェア。主にテストに用いられる

**イ.クロスコンパイラ**

→正しい。問題文はクロスコンパイラの説明です

ウ.最適化コンパイラ

→誤り。最適化コンパイラは通常のコンパイラよりもコードを最適化し、処理を高速にできるコンパイラです

エ.プログラムジェネレータ

→プログラムジェネレータは、処理条件を指定することでプログラムを生成するソフトです。

# 「出典：基本情報技術者平成23年秋期 午前問23」

静的テストツールの機能に分類されるものはどれか。

ア.ソースコードを解析して、プログラムの誤りを検出する。

イ.テスト対象モジュールに必要なドライバ又はスタブを生成する。

ウ.テストによって実行した経路から網羅度を算出する。

エ.プログラムの特定の経路をテストするためのデータを生成する。

# 「出典：基本情報技術者平成23年秋期 午前問23」

静的テストツールの機能に分類されるものはどれか。

**ア.ソースコードを解析して、プログラムの誤りを検出する。**

→正しい。静的テストツールは実行せずにソースコードを解析します

イ.テスト対象モジュールに必要なドライバ又はスタブを生成する。

→誤り、テスト環境設定ツールであり、静的テストツールではありません

ウ.テストによって実行した経路から網羅度を算出する。

→カバレッジモニタの説明で、動的テストツールです

エ.プログラムの特定の経路をテストするためのデータを生成する。

→テスト自動生成ツールであり、静的テストツールではありません

# 「出典：応用情報技術者平成22年秋期 午前問44」

プログラム実行中の特定の時点で成立する変数間の関係や条件を記述した論理式を埋め込んで、そのプログラムの正当性を検証する手法はどれか。

ア.アサーションチェック

イ.コード追跡

ウ.スナップショットダンプ

エ.テストカバレッジ分析

# 「出典：応用情報技術者平成22年秋期 午前問44」

プログラム実行中の特定の時点で成立する変数間の関係や条件を記述した論理式を埋め込んで、そのプログラムの正当性を検証する手法はどれか。

## ア.アサーションチェック

→正しい。アサーションチェックの説明です

## イ.コード追跡

→命令単位でプログラムを逐次実行して、プログラムの実行直後のメモリの状態を確認する手法です。トレーサなど

## ウ.スナップショットダンプ

→プログラムが実行されるたびに、メモリやレジスタの内容を出力します

## エ.テストカバレッジ分析

→テストの網羅率（どのプログラムを通ったか）を確認する手法です

# 「出典：応用情報技術者平成25年秋期 午前問21」

表はossのライセンスごとに、そのossを利用したプログラムを配布するとき、ソースコードを公開しなければならないかどうかを示す。a～dに入れるライセンスの適切な組合せはどれか。ここで、表中の"○"は公開しなければならないことを表し、"×"は公開しなくてもよいことを表す。

|                          | a | b | c | d |
|--------------------------|---|---|---|---|
| OSS のソースコードを修正して作ったプログラム | ○ | ○ | ○ | × |
| OSS に静的にリンクしたプログラム       | ○ | ○ | × | × |
| OSS に動的にリンクしたプログラム       | ○ | × | × | × |

|   | a    | b    | c   | d    |
|---|------|------|-----|------|
| ア | GPL  | LGPL | MPL | BSD  |
| イ | GPL  | MPL  | BSD | LGPL |
| ウ | LGPL | GPL  | MPL | BSD  |
| エ | LGPL | MPL  | BSD | GPL  |

# 「出典：応用情報技術者平成25年秋期 午前問21」

表はossのライセンスごとに、そのossを利用したプログラムを配布するとき、ソースコードを公開しなければならないかどうかを示す。a～dに入れるライセンスの適切な組合せはどれか。ここで、表中の"○"は公開しなければならないことを表し、"×"は公開しなくてもよいことを表す。

|                          | a | b | c | d |
|--------------------------|---|---|---|---|
| OSS のソースコードを修正して作ったプログラム | ○ | ○ | ○ | × |
| OSS に静的にリンクしたプログラム       | ○ | ○ | × | × |
| OSS に動的にリンクしたプログラム       | ○ | × | × | × |

|   | a    | b    | c   | d    |
|---|------|------|-----|------|
| ア | GPL  | LGPL | MPL | BSD  |
| イ | GPL  | MPL  | BSD | LGPL |
| ウ | LGPL | GPL  | MPL | BSD  |
| エ | LGPL | MPL  | BSD | GPL  |

→GPLはソースコードを修正したものもリンクしたのものも公開します。

LGPLはGPLを緩くしたもので動的リンクを非公開にできる

MPLはLGPLを緩くしたもので静的リンクを非公開にできる

BSDはすべて非公開にできる

正解はア